

Three Approaches Explained

Yes. This problem can be understood through **3 levels of approaches**:

1. **Brute Force** — 2 loops
2. **Better** — Prefix Frequency + Binary Search
3. **Optimal** — Sliding Window

Let's build them one by one.

1. Brute Force Approach — Generate every substring

Idea

For every starting index i , try every ending index j .

For each substring:

```
s[i...j]
```

Count how many distinct characters it contains.

If:

```
distinct characters == k
```

update the answer.

Example

```
s = "aabacbebebe"
k = 3
```

We generate:

```
a
aa
aab
aaba
aabac
...
a
ab
aba
abac
...
b
ba
bac
...
```

For every substring, check:

```
How many unique characters are present?
```

Brute force implementation

We can optimize the checking slightly by maintaining frequency while expanding j .

```
C++
class Solution {
public:
    int longestKSubstr(string &s, int k) {
        int n = s.size();
        int ans = -1;

        for (int i = 0; i < n; i++) {
            vector<int> freq(26, 0);
            int distinct = 0;

            for (int j = i; j < n; j++) {
                int index = s[j] - 'a';
```

```

        if (freq[index] == 0) {
            distinct++;
        }

        freq[index]++;

        if (distinct == k) {
            ans = max(ans, j - i + 1);
        }
    }
}

return ans;
};

```

Time Complexity

$O(N^2)$

Because:

```

i = 0 → j moves through N characters
i = 1 → j moves through N-1 characters
...

```

Total:

```

N + (N-1) + (N-2) + ...
≈  $O(N^2)$ 

```

Space:

$O(26) = O(1)$

2. Better Approach — Prefix Frequency + Binary Search

This approach is more interesting.

The idea is:

Instead of checking every possible substring directly, can we check whether a substring of a particular length exists that contains exactly k distinct characters?

For example:

Can we find a valid substring of length 7?

If yes:

```

Can we find length 8?
Can we find length 9?
...

```

If no:

Try smaller lengths.

So we can use **Binary Search on the answer**.

Step 1: Prefix frequency

Create prefix counts.

```
prefix[i][c]
```

means:

Number of occurrences of character c before index i .

For example:

```
s = "aabac"
```

For character 'a':

```
index:  0 1 2 3 4
```

```
string: a a b a c
prefix count of a:
before:
0 → 0
1 → 1
2 → 2
3 → 2
4 → 3
5 → 3
```

Then for any substring:

```
[l...r]
```

frequency of character c is:

```
prefix[r + 1][c] - prefix[l][c]
```

So we can count distinct characters in a substring.

Step 2: Check a particular length

Suppose:

```
length = mid
```

Try every substring of that length.

For each substring:

```
[l ... l + mid - 1]
```

Check all 26 characters.

If exactly k characters have frequency greater than 0, then this length is possible.

Important monotonicity observation

Binary search works only when the answer property is monotonic.

Here, this problem is a little subtle.

For "**exactly k distinct characters**", the property:

```
If length L is possible,
then every smaller length is possible
```

is actually **not guaranteed in the simple boolean sense required for straightforward binary search over lengths**.

So a direct binary search on:

```
exists substring of length L with exactly k distinct
```

is not a universally safe binary-search predicate.

For example, substring structure can make validity for one length not behave monotonically enough for a normal binary search.

Therefore, **Prefix Sum + Binary Search is not really a standard valid "better approach" for this exact problem**.

However, if you want to use prefix frequency to improve substring checking, you can still do this:

- Enumerate all (i, j)
- Use prefix frequencies to calculate distinct characters in $O(26)$

That gives:

```
 $O(N^2 \times 26)$ 
```

which is effectively:

```
 $O(N^2)$ 
```

So it does not asymptotically improve over the incremental brute force.

Prefix frequency code

```
C++
class Solution {
public:
    int longestKSubstr(string &s, int k) {
        int n = s.size();

        vector<vector<int>> prefix(n + 1, vector<int>(26, 0));

        for (int i = 0; i < n; i++) {
            prefix[i + 1] = prefix[i];
            prefix[i + 1][s[i] - 'a']++;
        }

        int ans = -1;

        for (int l = 0; l < n; l++) {
            for (int r = l; r < n; r++) {
                int distinct = 0;

                for (int ch = 0; ch < 26; ch++) {
                    int count =
                        prefix[r + 1][ch] - prefix[l][ch];

                    if (count > 0) {
                        distinct++;
                    }
                }

                if (distinct == k) {
                    ans = max(ans, r - l + 1);
                }
            }
        }

        return ans;
    }
};
```

Complexity:

Time: $O(N^2 \times 26) \approx O(N^2)$
 Space: $O(N \times 26)$

So honestly, this is **not better than the first approach** for this problem.

3. Optimal Approach — Sliding Window ★

This is the expected approach.

The key idea is to maintain a window:

```
[left ..... right]
```

such that:

```
distinct characters <= k
```

We expand **right**.

If distinct characters become greater than **k**, we shrink from **left**.

Whenever:

```
distinct == k
```

our current window is valid.

Then:

```
C++
ans = max(ans, right - left + 1);
```

Let's understand the logic deeply

Suppose:

```
s = "aabacbebebe"
k = 3
```

Initially:

```
left = 0
right = 0
distinct = 0
```

We expand **right**.

Add 'a'

[a]

Distinct:

{a} = 1

Less than k.

Continue.

Add another 'a'

[aa]

Frequency:

a = 2

Distinct is still:

1

Add 'b'

[aab]

Distinct:

{a, b} = 2

Add 'a'

[aaba]

Still:

{a, b} = 2

Add 'c'

[aabac]

Now:

{a, b, c} = 3

Exactly k.

So:

ans = 5

Add 'b'

[aabacb]

Still exactly 3:

{a, b, c}

ans = 6

Add 'e'

[aabacbe]

Now:

```
{a, b, c, e}
```

We have:

```
distinct = 4 > k
```

Invalid.

So shrink from the left.

Remove first 'a':

```
[abacbe]
```

Still:

```
{a,b,c,e} = 4
```

Remove another 'a':

```
[bacbe]
```

Still 4.

Remove 'b':

```
[acbe]
```

Still 4.

Remove 'a':

```
[cbe]
```

Now:

```
{c,b,e} = 3
```

Valid again.

Continue expanding.

Eventually:

```
[cbebebe]
```

Length:

```
7
```

Exactly:

```
{c, b, e}
```

Therefore:

```
answer = 7
```

Optimal Code

```
C++
class Solution {
public:
    int longestKSubstr(string &s, int k) {
        vector<int> freq(26, 0);
        int left = 0;
        int distinct = 0;
        int ans = -1;

        for (int right = 0; right < s.size(); right++) {
            // Add current character
            int index = s[right] - 'a';
            if (freq[index] == 0) {
                distinct++;
            }
        }
    }
};
```

```

    freq[index]++;

    // Too many distinct characters
    while (distinct > k) {

        int leftIndex = s[left] - 'a';
        freq[leftIndex]--;

        if (freq[leftIndex] == 0) {
            distinct--;
        }

        left++;
    }

    // Exactly k distinct characters
    if (distinct == k) {
        ans = max(ans, right - left + 1);
    }

    return ans;
}
};

```

The main intuition

The entire algorithm is based on this invariant:

Window always has at most K distinct characters

When we add **right**:

distinct <= k

→ window is okay.

When:

distinct > k

→ remove characters from the left until:

distinct <= k

And whenever:

distinct == k

→ calculate maximum length.

Why do we use while instead of if?

Suppose:

```

distinct = 5
k = 3

```

Removing one character may not reduce distinct count.

For example:

```

a a b c d e
↑
left

```

Removing one 'a':

a b c d e

We still have:

5 distinct

So we need:

```

C++
while (distinct > k)

```

not:

```
C++
if (distinct > k)
```

Visual representation

```
s = a a b a c b e b e b e
    ↑
    L

Initially expand R
```

When window has exactly k:

```

L           R
↓           ↓
a a b a c b
-----
Distinct = 3
Length = 6

answer = max(answer, 6)
```

Then:

```

L           R
↓           ↓
a a b a c b e
-----
Distinct = 4 ❌
```

Shrink:

```

a a b a c b e
↑
remove
a b a c b e
↑
remove
b a c b e
↑
remove
a c b e
↑
remove
c b e
↑
```

Now:

```

c b e
Distinct = 3 ✅
```

Continue expanding:

```

c b e b e b e
↑           ↑
L           R

Distinct = 3
Length = 7
```

Complexity comparison

Approach	Idea	Time	Space
Brute Force	Every start + end	$O(N^2)$	$O(1)$
Prefix Frequency	Prefix counts + all substrings	$O(26 \times N^2)$	$O(26N)$
Sliding Window ★	Expand + shrink dynamically	$O(N)$	$O(26)$

Since:

```
26 is constant
```

Optimal space is:

```
0(1)
```

and time:

$O(N)$

One important correction about the “Prefix Sum + Binary Search” approach

For this exact problem, a **normal binary search on substring length is not generally valid**, because the predicate:

"there exists a substring of length L with exactly K distinct characters"

is not guaranteed to be monotonic.

So the standard progression for this problem is more accurately:

```
Brute Force
↓
Better brute force with frequency tracking
↓
Optimal Sliding Window
```

The **sliding window** is the real optimal pattern here and is the one you should remember for interviews.

Pattern to remember:

```
Longest Substring + Exactly K Distinct Characters
      ↓
    Variable Sliding Window
      ↓
Expand right
      ↓
distinct > k ?
      ↓ yes
Shrink left
      ↓
distinct == k ?
      ↓ yes
Update answer
```

ChatGPT can make mistakes. Check important info.